

<HLS Programming with FPGAs>

The HLS Book

Chapter 3

CORDIC 1

Choi, Young-kyu

Learning Objectives

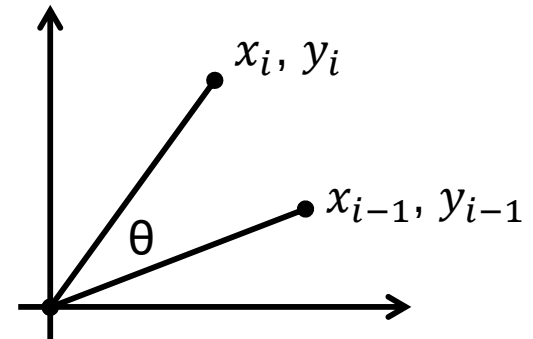
- Explain CORDIC method
- Implement CORDIC in C

CORDIC

- **C**Oordinate **R**otation **D**igital **C**omputer
- Efficient method to calculate trigonometric functions
- Replaces multiplication with addition, subtraction, shift, and table lookups

Rotation

$$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_{i-1} \\ y_{i-1} \end{bmatrix} = \begin{bmatrix} x_i \\ y_i \end{bmatrix}$$



Rotation matrix: $R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$

Computation: $x_i = x_{i-1} \cos \theta - y_{i-1} \sin \theta$

$$y_i = x_{i-1} \sin \theta + y_{i-1} \cos \theta$$

→ Can we get rid of multiplication?

Rotation without Multiplication

- Example) Rotation by 90 degrees ($\pi/2$)

$$R(90^\circ) = \begin{bmatrix} \cos 90^\circ & -\sin 90^\circ \\ \sin 90^\circ & \cos 90^\circ \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

$$\begin{aligned} x_i &= x_{i-1} \cos 90^\circ - y_{i-1} \sin 90^\circ \\ &= x_{i-1} \cdot 0 - y_{i-1} \cdot 1 \\ &= -y_{i-1} \end{aligned}$$

$$\begin{aligned} y_i &= x_{i-1} \sin 90^\circ + y_{i-1} \cos 90^\circ \\ &= x_{i-1} \cdot 1 + y_{i-1} \cdot 0 \\ &= x_{i-1} \end{aligned}$$

Rotation without Multiplication

- Example) Rotation by 45 degrees ($\pi/4$)

$$R(45^\circ) = \begin{bmatrix} \cos 45^\circ & -\sin 45^\circ \\ \sin 45^\circ & \cos 45^\circ \end{bmatrix} = \begin{bmatrix} \sqrt{2}/2 & -\sqrt{2}/2 \\ \sqrt{2}/2 & \sqrt{2}/2 \end{bmatrix}$$

$$\begin{bmatrix} \sqrt{2}/2 & -\sqrt{2}/2 \\ \sqrt{2}/2 & \sqrt{2}/2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \sqrt{2}/2x - \sqrt{2}/2y \\ \sqrt{2}/2x + \sqrt{2}/2y \end{bmatrix} \quad \text{: still needs multiplication ☹}$$

– Using simpler rotation matrix:

$$\text{Ex) } R() = \begin{bmatrix} 1/2^0 & -1/2^0 \\ 1/2^0 & 1/2^0 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \text{ and scale by } \sqrt{2}/2$$

$$\text{Ex) } R() = \begin{bmatrix} 1/2^1 & -1/2^1 \\ 1/2^1 & 1/2^1 \end{bmatrix} \text{ and scale by } \sqrt{2},$$

Rotation without Multiplication

- Generalization

- Consider rotation matrix:

$$R_i(\theta) = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i) \end{bmatrix}$$

- Using trigonometric characteristics,

$$\cos(\theta_i) = \frac{1}{\sqrt{1 + \tan^2(\theta_i)}} \quad \sin(\theta_i) = \frac{\tan(\theta_i)}{\sqrt{1 + \tan^2(\theta_i)}}$$

- Rotation matrix can be re-written as:

$$R_i = \frac{1}{\sqrt{1 + \tan^2(\theta_i)}} \begin{bmatrix} 1 & -\tan(\theta_i) \\ \tan(\theta_i) & 1 \end{bmatrix}$$

- Restrict $\tan(\theta_i)$ to be power of 2's (shift operation):

$$\tan(\theta_i) = 2^{-i}.$$

- Then the rotation matrix becomes:

$$v_i = K_i \begin{bmatrix} 1 & -2^{-i} \\ 2^{-i} & 1 \end{bmatrix} \begin{bmatrix} x_{i-1} \\ y_{i-1} \end{bmatrix} \quad \text{where} \quad K_i = \frac{1}{\sqrt{1 + 2^{-2i}}}$$

- Further generalization to enable negative rotation:

$$v_i = K_i \begin{bmatrix} 1 & -\sigma_i 2^{-i} \\ \sigma_i 2^{-i} & 1 \end{bmatrix} \begin{bmatrix} x_{i-1} \\ y_{i-1} \end{bmatrix} \quad \text{where } \sigma_i \text{ is 1 if } (\theta_i) \text{ is positive}$$

σ_i is -1 if (θ_i) is negative

- After series of iterations, scaling factor becomes:

$$K(n) = \prod_{i=0}^{n-1} K_i = \prod_{i=0}^{n-1} \frac{1}{\sqrt{1 + 2^{-2i}}}$$

– Processing (CORDIC) gain: $A = \frac{1}{K}$, where $K = \prod_{i=0}^{n-1} \frac{1}{\sqrt{1 + 2^{-2i}}}$

i	2^{-i}	Rotating Angle	Scaling Factor	CORDIC Gain
0	1.0	45.000°	1.41421	1.41421
1	0.5	26.565°	1.11803	1.58114
2	0.25	14.036°	1.03078	1.62980
3	0.125	7.125°	1.00778	1.64248
4	0.0625	3.576°	1.00195	1.64569
5	0.03125	1.790°	1.00049	1.64649
6	0.015625	0.895°	1.00012	1.64669

- CORDIC gain converges to:

$$A = \frac{1}{K} = \lim_{n \rightarrow \infty} \prod_{i=0}^{n-1} \sqrt{1 + 2^{-2i}} \approx 1.64676025812107$$

3.3 Calculating Sine and Cosine

- Example) Calculate $\sin 60^\circ$ and $\cos 60^\circ$ (in 5 iterations)
 - Step 1) Determine rotation by positive or negative angle
 - Start at 0°
 - 1st iteration: Rotate by positive angle (toward 60°)

– Step 2) Perform rotation

$$\begin{pmatrix} \cos 45^\circ \\ \sin 45^\circ \end{pmatrix} = v_0 * \begin{pmatrix} \cos 0^\circ \\ \sin 0^\circ \end{pmatrix}$$

- 1st iteration:

$$\text{– Rotation matrix } v_0 = \frac{1}{\sqrt{1 + 2^{-2*0}}} \begin{pmatrix} 1 & -2^{-0} \\ 2^{-0} & 1 \end{pmatrix}$$

$$\text{– } \cos(45^\circ) = [\cos(0^\circ) * 1 - \sin(0^\circ) * (+1) * 2^{-0}] / 1.41421$$

$$\text{– } \sin(45^\circ) = [\cos(0^\circ) * (+1) * 2^{-0} + \sin(0^\circ) * 1] / 1.41421$$

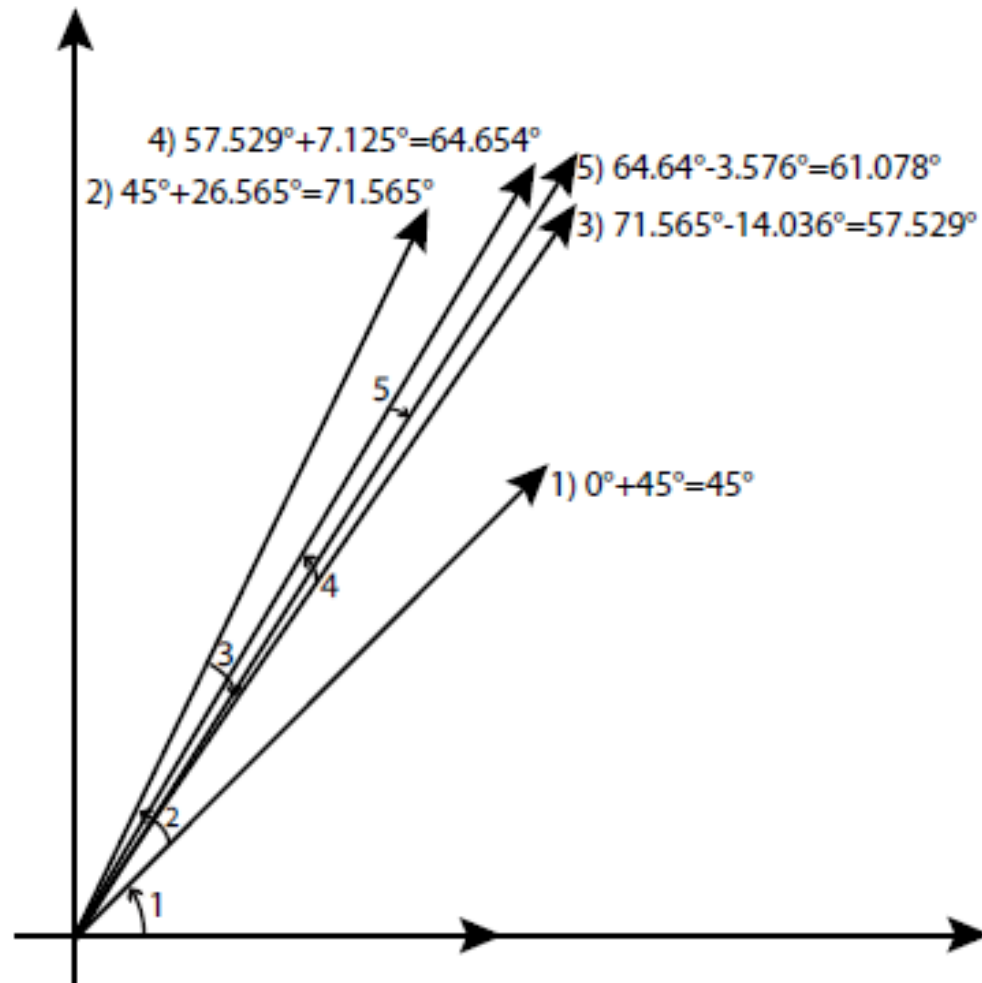
$$(1.41421 = \sqrt{1 + 2^{-2*0}})$$

3.3 Calculating Sine and Cosine

- Example) Calculate $\sin 60^\circ$ and $\cos 60^\circ$ (in 5 iterations)
 - Step 3) If iteration < 5 , go to step 1, else go to step 4.
 - Step 4) Multiply reciprocal of CORDIC gain for 5 iterations ($1/1.64649 = 0.60735$)

i	2^{-i}	Rotating Angle	Scaling Factor	CORDIC Gain
0	1.0	45.000°	1.41421	1.41421
1	0.5	26.565°	1.11803	1.58114
2	0.25	14.036°	1.03078	1.62980
3	0.125	7.125°	1.00778	1.64248
4	0.0625	3.576°	1.00195	1.64569
5	0.03125	1.790°	1.00049	1.64649
6	0.015625	0.895°	1.00012	1.64669

– Graphic representation:



Exercise

- Compute $\cos(45^\circ + 26.565^\circ)$, assuming $\cos(45^\circ)$ and $\sin(45^\circ)=0.70711$. Write down the whole solution equation (similar to step 2 in computing $\cos 45^\circ$). Check if the result of your computation matches the value of $\cos(71.565^\circ)$.

i	2^{-i}	Rotating Angle	Scaling Factor	CORDIC Gain
0	1.0	45.000°	1.41421	1.41421
1	0.5	26.565°	1.11803	1.58114
2	0.25	14.036°	1.03078	1.62980
3	0.125	7.125°	1.00778	1.64248
4	0.0625	3.576°	1.00195	1.64569
5	0.03125	1.790°	1.00049	1.64649
6	0.015625	0.895°	1.00012	1.64669

Solution to Exercise

- $\cos(45^\circ + 26.565^\circ)$
= $[\cos(45^\circ) - \sin(45^\circ) * (+1) * 2^{-1}] / 1.11803$
= 0.31623 (1.11803 = $\sqrt{1 + 2^{-2*1}}$)
= $\cos(71.565^\circ)$

CORDIC code

```
#define NUM_ITERATIONS 5

// The file cordic.h holds definitions for the data types and constant values
#include "cordic.h"

// The cordic_phase array holds the angle for the current rotation
THETA_TYPE cordic_phase[NUM_ITERATIONS] = {0.785, 0.463,
0.245, 0.124, 0.0624}; // 45°, 26.565°, .... in radian

void cordic(THETA_TYPE theta, COS_SIN_TYPE &s, COS_SIN_TYPE &c)
{
    // Set the initial vector that we will rotate
    // current_cos = I; current_sin = Q
    COS_SIN_TYPE current_cos = 0.60735;
    COS_SIN_TYPE current_sin = 0.0;

```

CORDIC gain: 1.64649
 $\rightarrow 1/1.64649 = 0.60735$

$$\begin{bmatrix} 1 & -\sigma_i 2^{-i} \\ \sigma_i 2^{-i} & 1 \end{bmatrix}$$

```
COS_SIN_TYPE factor = 1.0;
// This loop iteratively rotates the initial vector to find the
// sine and cosine values corresponding to the input theta angle
for (int j = 0; j < NUM_ITERATIONS; j++) {
    // Determine if we are rotating by a positive or negative angle
    int sigma = (theta < 0) ? -1 : 1;

    // Multiply previous iteration by 2^(-j)
    COS_SIN_TYPE cos_shift = current_cos * sigma * factor;
    COS_SIN_TYPE sin_shift = current_sin * sigma * factor;

    // Perform the rotation
    current_cos = current_cos - sin_shift;
    current_sin = current_sin + cos_shift;

    // Determine the new theta
    theta = theta - sigma * cordic_phase[j];

    factor = factor / 2;
}

// Set the final sine and cosine values
s = current_sin; c = current_cos;
}
```

<Figure 3.3 of HLS book>

Make Vitis HLS Project for CORDIC

- Copy the code in Figure 3.3 to `cordic.cpp`
- Using Vitis HLS GUI (or command line script), make a new project named “cordic” and add `cordic.cpp`
- Set clock period = 4ns, uncertainty = 1.08ns, default part selection of `xcvu9p-flga2104-1-e`
- Set all types to “float”

Testbench Generation

- Generate multiple theta values from $-\pi/2 \sim +\pi/2$
- Compare sine & cosine values computed from CORDIC kernel ($\sin_{CORDIC}$, $\cos_{CORDIC}$) with math.h library calls ($\sin_{MATH}$, $\cos_{MATH}$), and measure error

```
for( int i = 0; i < TEST_NUM ; i++ )
{
    float theta = -M_PI/2 + M_PI/TEST_NUM*i;
    float cos_math = cos(theta);
    float sin_math = sin(theta);

    COS_SIN_TYPE sin_cordic, cos_cordic;

    cordic(theta, sin_cordic, cos_cordic);

    float sin_error = fabs(sin_math-sin_cordic);
    float cos_error = fabs(cos_math-cos_cordic);
}
```

- Error metrics:
 - Maximum error: $\max(|\sin_{CORDIC} - \sin_{MATH}|)$
 - RMSE: $\sqrt{\frac{\sum (\sin_{CORDIC} - \sin_{MATH})^2}{N}}$
 - SQNR: $10 * \log_{10}(\frac{\sum (\sin_{CORDIC})^2}{\sum (\sin_{CORDIC} - \sin_{MATH})^2})$

- CORDIC error measurement result with C Sim:
 - Tested with 10000 values in range of $-\pi/2 \sim \pi/2$
 - Output maximum error: 0.0627
 - Output RMSE: 0.0245
 - Output SQNR: 29.2dB

Vitis HLS Synthesis Result

Modules & Loops	Issu Viol	Di	Slack	Latency	Latency(Iteration	Latency Interval	Trip Count	Pipelined		
▼ cordic	ii	II	-	79	316.000	-	80	-	no	
CORDIC_LOOP	ii	II	Merr 1	-	77	308.000	18	15	5	yes

BRAM	DSP	FF	LUT	URAM
0	10	1425	975	0
-	-	-	-	-

The II Violation in module 'cordic' (loop 'CORDIC_LOOP'): Unable to enforce a carried dependence constraint (II = ('current_sin_write_ln25', [cordic_float.cpp:25](#)) of variable 'current_sin', [cordic_float.cpp:25](#) on local variable 'current_sin', [cordic_float.cpp:25](#) on local variable 'current_sin'.

Vitis HLS Synthesis Result

▼ SCHEDULE

⚠ [HLS 200-880]

[LINK](#)

The II Violation in module 'cordic' (loop 'CORDIC_LOOP'): Unable to enforce a carried dependence constraint (II = 1, d ('theta', [cordic.cpp:30](#)) and 'icmp' operation ('icmp_ln22', [cordic.cpp:22](#)).

⚠ [HLS 200-880]

[LINK](#)

The II Violation in module 'cordic' (loop 'CORDIC_LOOP'): Unable to enforce a carried dependence constraint (II = 2, d ('theta', [cordic.cpp:30](#)) and 'icmp' operation ('icmp_ln22', [cordic.cpp:22](#)).

```
// for (int j = 0; j < NUM_ITERATIONS; j++) {  
  // Determine if we are rotating by a positive or negative angle  
  int sigma = (theta < 0) ? -1 : 1;  
  
  // Multiply previous iteration by 2^(-j)  
  COS_SIN_TYPE cos_shift = current_cos * sigma * factor;  
  COS_SIN_TYPE sin_shift = current_sin * sigma * factor;  
  
  // Perform the rotation  
  current_cos = current_cos - sin_shift;  
  current_sin = current_sin + cos_shift;  
  
  // Determine the new theta  
  theta = theta - sigma * cordic_phase[j];  
  
  factor = factor / 2;  
}
```

Problem 1 : Loop-carried dependence

Problem 2 : Multiplication still exists

Summary

- CORDIC is an efficient method to calculate trigonometric functions without multiplication
- Implemented CORDIC algorithm in C
- Floating-point implementation of CORDIC has a poor performance